LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 – Enkapsulasi



Aditya Dwi Aryanto

103132400027

SD-01-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Enkapsulasi dalam pemrograman berorientasi objek merupakan konsep pembungkusan data dan method ke dalam satu kesatuan unit untuk menyembunyikan detail implementasi internal (data hiding). Di dalam Java, aksesibilitas terhadap anggota kelas diatur menggunakan access modifier seperti public, protected, private, maupun default. Secara umum, atribut diberi tingkat akses private agar tidak dapat diakses atau diubah secara langsung dari luar kelas. Sebagai gantinya, disediakan method dengan tingkat akses public, yaitu setter untuk mengubah data dan getter untuk mengambil nilai data tersebut.

Selain access modifier, Java juga menyediakan jenis modifier lain seperti final, static, abstract, dan synchronized untuk mengontrol perilaku kelas, method, maupun variabel. Untuk menginstansiasi objek dan menginisialisasi atributnya, digunakan method khusus yang dinamakan constructor. Constructor memiliki nama yang sama persis dengan nama kelasnya dan tidak memiliki tipe kembalian. Java juga mendukung pembuatan beberapa constructor dalam satu kelas melalui konsep overloading. Sementara itu, package digunakan untuk mengelompokkan dan mengorganisasi kelas-kelas yang berkaitan agar struktur program lebih rapi serta terhindar dari bentrok penamaan kelas atau method.

B. Guided

1. Encapsulasi

encapsulasi.java

```
package aditya.modul_4_encapsulasi;


public class Modul_4_encapsulasi {


    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

manusia.java

```
package aditya.modul_4_encapsulasi.Manusia;

public class Manusia {
```

```
    private String nama;
    private int umur;
//definisi constructor

    public Manusia() {
    } //constructor pertama = default tanpa parameter

    public Manusia(String a) { //constructor kedua
        nama = a;
    }

    public Manusia(String a, int b) { //constructor ketiga
        nama = a;
        umur = b;
    }
//definisi Method

    public void setNama(String a) {
        nama = a;
    }
    public String getNama() {
        return nama;
    }

    public void setUmur(int a) {
        umur = a;
    }

    public int getUmur() {
        return umur;
    }
}
```

Demonmanusia.java

```
package aditya.modul_4_encapsulasi.main;

import aditya.modul_4_encapsulasi.Manusia.Manusia;

public class DemonManusia {

    public static void main(String[] args) { // program utama
        Manusia arrMns[] = new Manusia[3]; // buat array of Object
        Manusia objMns1 = new Manusia(); // constructor pertama
```

```
        objMns1.setNama("Markonah");
        objMns1.setUmur(76);
//constructor kedua

        Manusia objMns2 = new Manusia("Mat Conan");

//constructor ketiga
        Manusia objMns3 = new Manusia("Bajuri", 13);
        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;
        for (int i = 0; i < 3; i++) {

            System.out.println("Nama : " + arrMns[i].getNama());
            System.out.println("Umur : " + arrMns[i].getUmur());
            System.out.println();
        }
    }
}
```

Output :

```
Nama : Markonah
Umur : 76

Nama : Mat Conan
Umur : 0

Nama : Bajuri
Umur : 13

------------------------
BUILD SUCCESS
```

2. Token

hargapulsa.java

```
package hargapulsa;
```

```
public class hargapulsa {

   public void info() {
      System.out.println("Harga Pulsa");
      System.out.println("Rp. 5000");
      System.out.println("Rp. 10000");
      System.out.println("Rp. 20000");
      System.out.println("Rp. 50000");
      System.out.println("Rp. 100000");
   }
}
```

hargatoken.java

```
package hargatoken;

public class hargatoken {

   public void info() {
      System.out.println("Harga Token");
      System.out.println("Rp. 20000");
      System.out.println("Rp. 50000");
      System.out.println("Rp. 100000");
      System.out.println("Rp. 200000");
      System.out.println("Rp. 500000");
   }
}
```

Main.java

```
package main;

import hargatoken.hargatoken;
import hargapulsa.hargapulsa;

public class main {

   public static void main(String[] args) {
      hargatoken objectToken = new hargatoken();
      objectToken.info();
      hargapulsa objectPulsa = new hargapulsa();
      objectPulsa.info();
   }
```

```
}
```

Output :

```
] --- exec:3.6.3:exec (default-cli) @ Modul_4_encapsulasi ---
  Harga Token
  Rp. 20000
  Rp. 50000
  Rp. 100000
  Rp. 200000
  Rp. 500000
  Harga Pulsa
  Rp. 5000
  Rp. 10000
  Rp. 20000
  Rp. 50000
- Rp. 100000
  ------------------------------------------------------------
  BUILD SUCCESS
```

C. Unguided

1. Soal Unguided 1

Kamu sedang membangun program kecil untuk mencatat dan memeriksa kualitas dataset sebelum dianalisis, seperti tahap awal pengecekan *missing value*. Setiap dataset punya nama, jumlah baris, jumlah kolom, dan jumlah sel yang kosong (*missing*).

Jawaban :

Datase.java

```
package unguided.model;


public class Dataset {
```

```
public static final double BATAS_MISSING = 5.0;



private static int totalDataset = 0;



private String nama;
private int jumlahBaris;
private int jumlahKolom;
private int jumlahMissing;



public Dataset() {
    this("Tanpa Nama");
}



public Dataset(String nama) {
    this(nama, 0, 0, 0);
}



public Dataset(String nama, int jumlahBaris, int jumlahKolom, int jumlahMissing) {
    this.nama = nama;
    setJumlahBaris(jumlahBaris);
    setJumlahKolom(jumlahKolom);
```

```
    setJumlahMissing(jumlahMissing);
    totalDataset++;
}


public static int getTotalDataset() {
    return totalDataset;
}


public String getNama() {
    return nama;
}

public int getJumlahBaris() {
    return jumlahBaris;
}

public int getJumlahKolom() {
    return jumlahKolom;
}

public int getJumlahMissing() {
    return jumlahMissing;
}


public void setNama(String nama) {
```

```
    this.nama = nama;
}


public void setJumlahBaris(int jumlahBaris) {
    if (jumlahBaris >= 0) {
        this.jumlahBaris = jumlahBaris;
    }
}

public void setJumlahKolom(int jumlahKolom) {
    if (jumlahKolom >= 0) {
        this.jumlahKolom = jumlahKolom;
    }
}

public void setJumlahMissing(int jumlahMissing) {
    if (jumlahMissing >= 0) {
        this.jumlahMissing = jumlahMissing;
    }
}


public double getPersentaseMissing() {
    long totalSel = (long) jumlahBaris * jumlahKolom;
    if (totalSel == 0) {
        return 0;
    }
```

```
        return (double) jumlahMissing / totalSel * 100;
    }



    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}
```

LaporanDataset.java

```
package unguided.Laporan;

import java.util.Locale;
import unguided.model.Dataset;

public class LaporanDataset {

    public static void cetak(Dataset d) {
        String status = d.perluDibersihkan() ? "Perlu dibersihkan" : "Bersih";

        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama : " + d.getNama());
        System.out.println("Jumlah Baris : " + d.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + d.getJumlahKolom());
        System.out.println(String.format(Locale.US, "Missing : %d sel (%.2f%%)",
                d.getJumlahMissing(), d.getPersentaseMissing()));
        System.out.println("Status : " + status);
        System.out.println();
    }
}
```

Main.java

```
package unguided.Main;

import unguided.Laporan.LaporanDataset;
import unguided.model.Dataset;
```

```
public class Main {

    public static void main(String[] args) {

        Dataset d1 = new Dataset();
        d1.setNama("Titanic");
        d1.setJumlahBaris(891);
        d1.setJumlahKolom(12);
        d1.setJumlahMissing(866);


        Dataset d2 = new Dataset("Wine Quality");


        Dataset d3 = new Dataset("Iris", 150, 5, 0);


        Dataset[] daftar = {d1, d2, d3};
        for (Dataset d : daftar) {
            LaporanDataset.cetak(d);
        }

        System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
    }
}
```

Output :

```
=== Laporan Dataset ===
Nama : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing : 866 sel (8.10%)
Status : Perlu dibersihkan

=== Laporan Dataset ===
Nama : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing : 0 sel (0.00%)
Status : Bersih

=== Laporan Dataset ===
Nama : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing : 0 sel (0.00%)
Status : Bersih

Total dataset dibuat : 3
------------------------------------------------
BUILD SUCCESS
```

Penjelasan :

Program ini terdiri dari tiga class di tiga package. Class Dataset (package model) menyimpan nama, jumlah baris, jumlah kolom, dan jumlah missing sebagai atribut private, sehingga hanya bisa diakses lewat getter dan setter. Inilah yang disebut enkapsulasi. Setter angka menolak nilai negatif supaya data tidak rusak. Ada juga konstanta BATAS_MISSING bernilai 5.0 dan variabel static totalDataset yang bertambah setiap objek dibuat. Nilainya bisa diambil lewat Dataset.getTotalDataset() tanpa membuat objek. Tiga constructor dibuat dengan overloading, dan dua yang sederhana memanggil constructor lengkap lewat this(...), jadi hitungan total tetap bertambah satu kali per objek.

Method getPersentaseMissing() menghitung sel kosong dibagi total sel (baris × kolom) lalu dikali 100, dan mengembalikan 0 kalau total sel nol agar tidak terjadi pembagian dengan nol. Method perluDibersihkan() membandingkan persentase itu dengan batas 5%, dan hasilnya true kalau melebihi.

Class LaporanDataset (package laporan) hanya bertugas mencetak laporan satu objek Dataset dengan format yang diminta soal. Statusnya "Perlu dibersihkan" atau "Bersih" tergantung hasil perluDibersihkan(). Class Main (package main) membuat tiga objek dengan tiga constructor berbeda, menyimpannya dalam array, mencetak laporannya dengan perulangan for, lalu menampilkan total dataset yang dibuat.

D. Kesimpulan

- Penerapan Enkapsulasi: Enkapsulasi diimplementasikan dengan menyembunyikan detail atribut (data hiding) menggunakan access modifier private, serta menyediakan method interface bertipe public (getter dan setter) untuk mengakses dan mengubah data secara aman.
- Penggunaan Modifier: Modifier di Java diatur untuk menentukan hak akses serta karakteristik dari kelas, method, dan variabel.
- Peran Constructor: Constructor berfungsi menginstansiasi objek dan menginisialisasi atribut. Java mendukung constructor overloading, yang memungkinkan pembuatan beberapa constructor dengan kombinasi parameter yang berbeda.
- Pengorganisasian dengan Package: Package membantu mengelompokkan kelas-kelas terkait, menjaga kerapian struktur direktori program, serta mencegah konflik atau bentrok penamaan kelas/method.